

# An Overview of Flow Matching and Score matching models

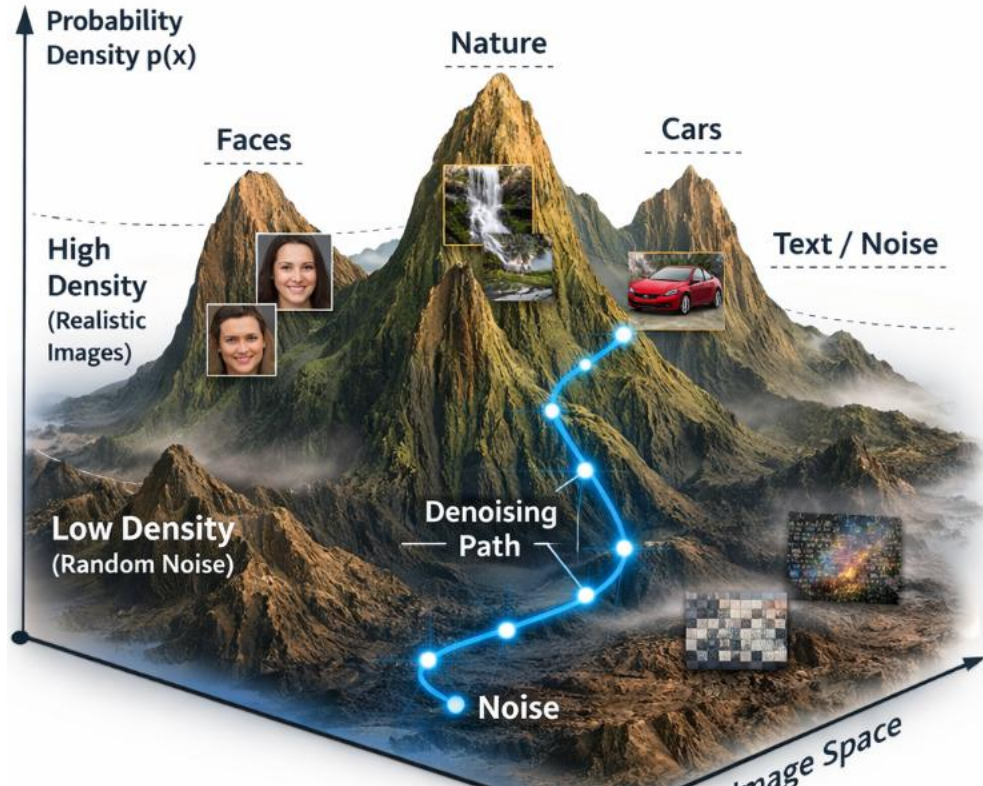
***Instructor: Byungchul Kim***  
[bc.kim@skku.edu](mailto:bc.kim@skku.edu)

The contents of this lecture note are adapted from Holderrieth and Erives (2025), *An Introduction to Flow Matching and Diffusion Models*

# Our goal: Find a way to sample from $p_{\text{data}}(o_t)$

We want to “generate” data by sampling it from the data distribution

Probability Landscape of Image Space



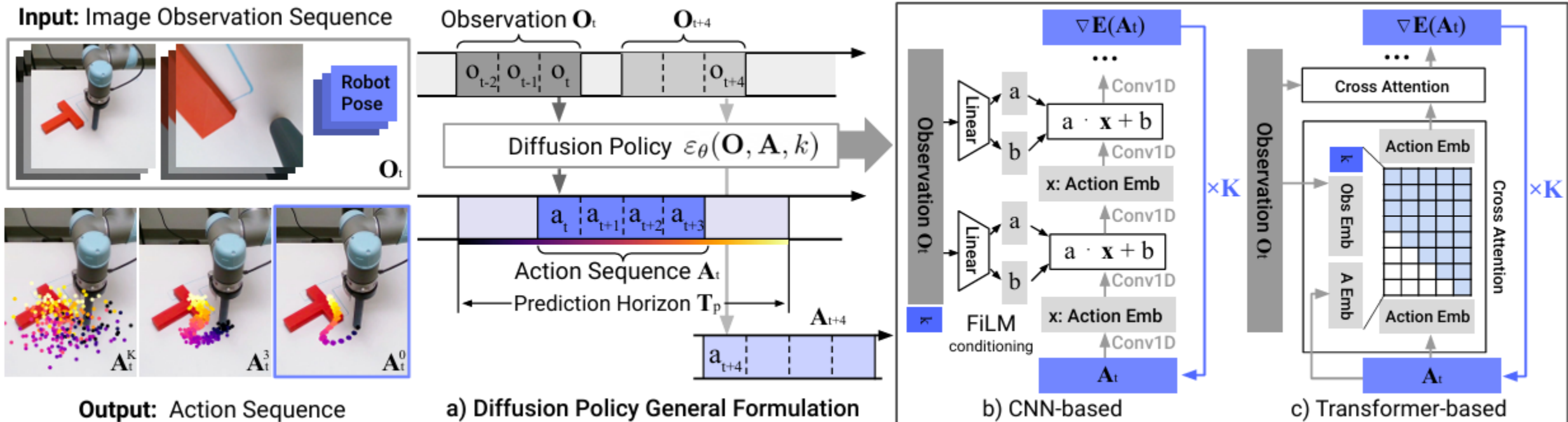
- We can imagine probability density function,  $p_{\text{data}}(x)$ , in high dimension space as shown in left.
- High value in  $p_{\text{data}}(x)$  regions will have realistic images while the low value regions will have random noise.
- Estimating  $p_{\text{data}}(x)$  with model parameter  $\theta$  can be useful in generative AI:

$$p_{\text{data}}(x) \approx p_{\theta}(x)$$

Can we **sample data** from  $p_{\theta}(x)$ , which **approximates**  $p_{\text{data}}(x)$ ?  
(1) approximation, (2) sample from it

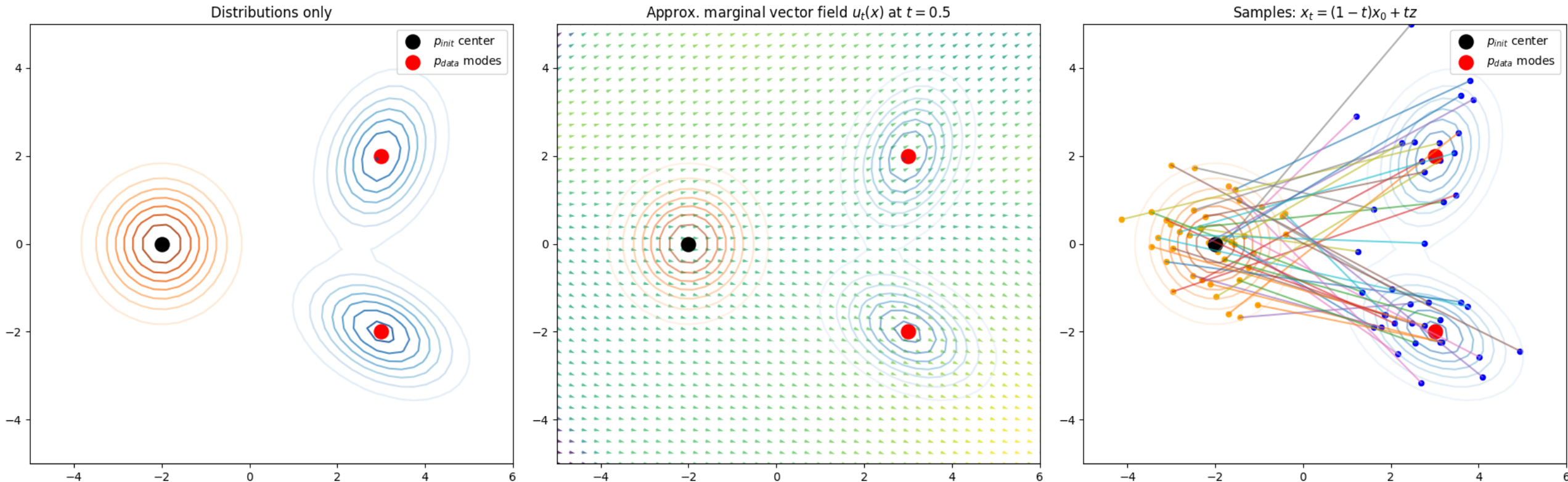
# Diffusion Policy – Using DDPM for Imitation Learning

Instead of generating image, diffusion policy outputs the action sequence



# Strategy: Transforming known distribution to other distribution

Flow matching: Learning a time-dependent “**vector field**” that transports an **initial distribution** to a **target data distribution**



# Problem Definition

---

We use ordinary differential equations (ODEs) for the transportation

- Assume a point in  $d$ -dimension that moves from time 0 to 1

$$x_t \in \mathbb{R}^d \text{ where } X_t: [0, 1] \rightarrow \mathbb{R}^d, t \mapsto X(t) = X_t$$

- Vector field to define the ODE: Given position  $x_t$  and time  $t$ , it determines the velocity.

$$u: \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, (x, t) \mapsto u_t(x)$$

- Formalization of the vector field ( $u_t$ ): Velocity is time derivative of position

$$\frac{d}{dt} X_t = u_t(X_t) \text{ where, } X_0 = x_0$$

- Definition of flow ( $\psi_t$ ): Position of  $X_t$  obtained from its initial position  $x_0$ . i.e.,  $X_t = \psi_t(x_0)$

$$\frac{d}{dt} \psi_t(x_0) = u_t(\psi_t(x_0)) \text{ where, } \psi_0(x_0) = x_0$$

# Visualization of flow $\psi_t$ and vector field $u_t$

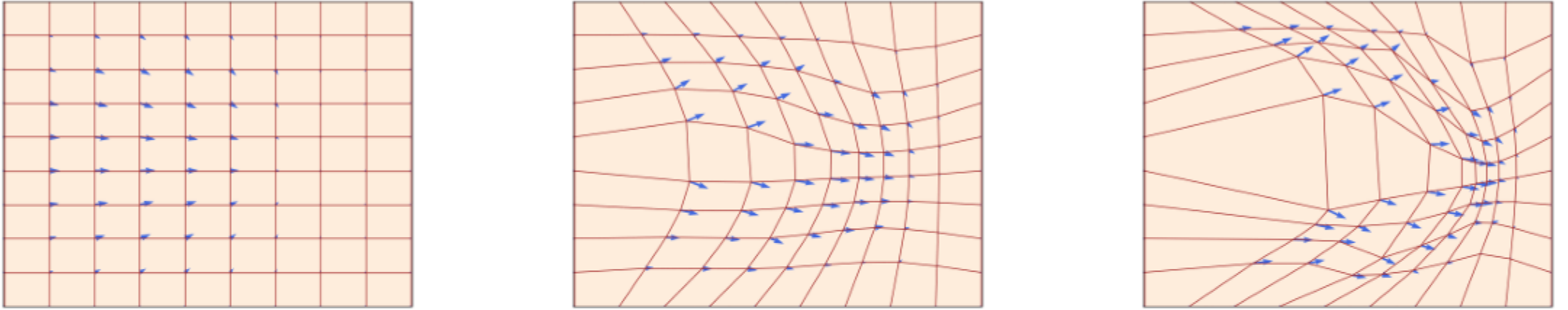


Figure 1: A flow  $\psi_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$  (red square grid) is defined by a velocity field  $u_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$  (visualized with blue arrows) that prescribes its instantaneous movements at all locations (here,  $d = 2$ ). We show three different times  $t$ . As one can see, a flow is a diffeomorphism that "warps" space. Figure from [26].

➤ **Vector field  $u_t$  deforms the grid (flow  $\psi_t$ ) and change the distribution.**

# Example: Linear vector field

$$\frac{d}{dt}\psi_t(x_0) = u_t(\psi_t(x_0)) \text{ where, } \psi_0(x_0) = x_0$$

**Note:**  $\psi_t(x_0)$  is flow (position) and  $u_t(x)$  is vector field (velocity)

➤ Assume we have a vector field

$$u_t(x) = -\theta x$$

➤ We can obtain the flow  $\psi_t$  from definition of vector field as

$$\frac{d}{dt}X_t = u_t(X_t) \text{ where, } X_0 = x_0$$

$$\frac{d}{dt}X_t = -\theta x$$

$$X_t = e^{-\theta t}x_0 = \psi_t(x_0)$$

➤ We can check the flow  $\psi_t$  is correct by checking:

$$\frac{d}{dt}\psi_t(x_0) = -\theta e^{-\theta t}x_0 = -\theta\psi_t(x_0) = u_t(\psi_t(x_0))$$

# Flow model: A neural network vector field $u_t^\theta(x)$

---

- **Random initialization** (usually,  $p_{init}$  is set to be a normal distribution for easier sampling)

$$X_0 \sim p_{init}$$

- **ODE**

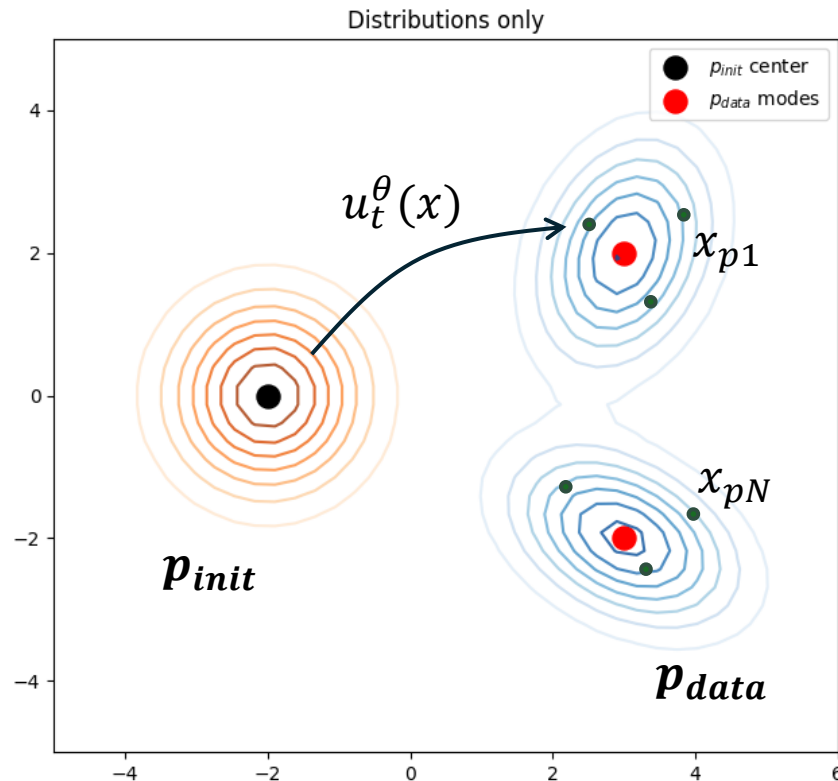
$$\frac{d}{dt}X_t = u_t^\theta(X_t)$$

- **Our goal:**

$$X_1 = \psi_1^\theta(X_0) \sim p_{data}$$

$$\text{Where, } X_0 \sim p_{init}$$

# So, how can we find a neural network $u_t^\theta(x)$ from collected dataset



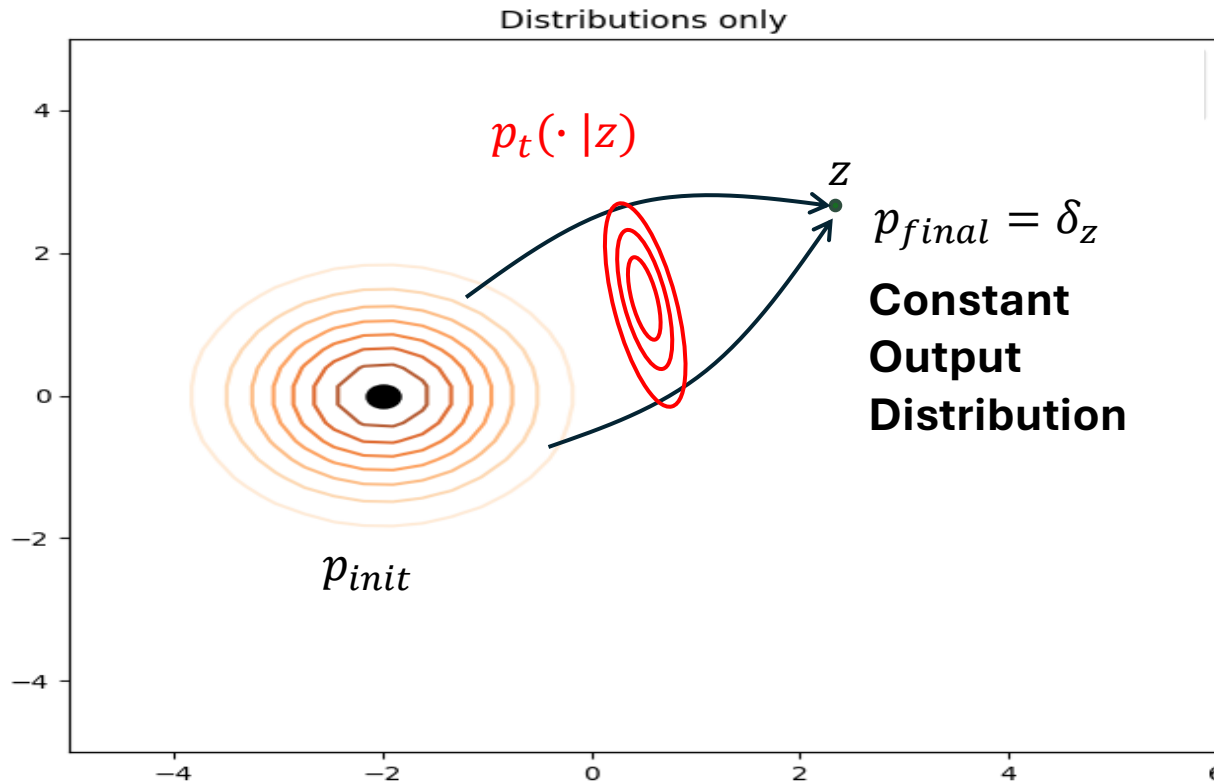
➤ Given collected data  $x_{p1}, x_{p2}, \dots, x_{pN}$ , we want to obtain  $u_t^\theta(x)$  that transform the distribution.

## ➤ Formulation

- Initial distribution:  $X_0 \sim p_{init}$
- Vector field at  $X_t$ :  $dX_t = u_t^\theta(X_t)dt$
- Final distribution:  $X_1 \sim p_{data}$

# Step 1: expressing the conditional probability ( $p_t(\cdot | z)$ ) path

Given data point  $z$ , what is the distribution at time  $t \in [0,1]$ ?



➤ Given a data point  $z$ , what is the conditional distribution at time  $t \in [0,1]$ ?

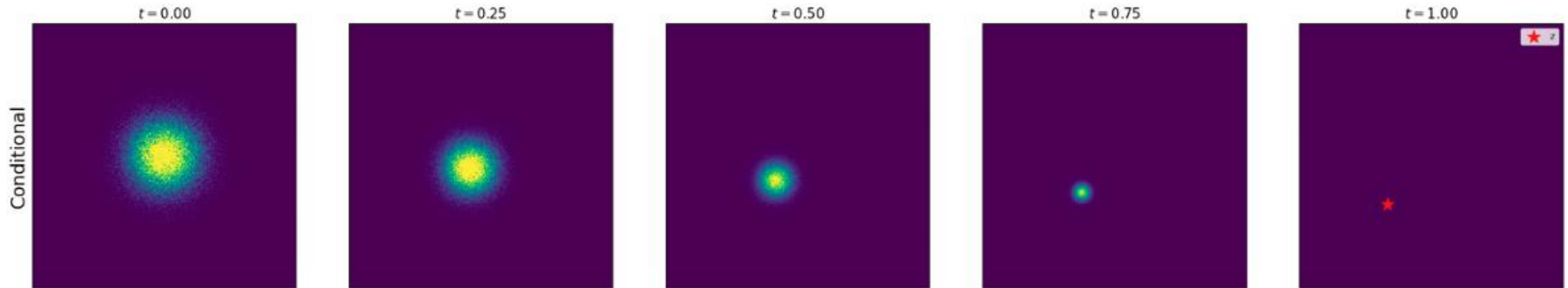
- $p_0(\cdot | z) = p_{init}$
- $p_1(\cdot | z) = p_{final} = \delta_z$
- At  $t = 0$ , the distribution is normal distribution and at  $t = 1$ , the distribution contracts to a single point  $z$ .

➤ We can first start from the conditional probability and then marginalize it.

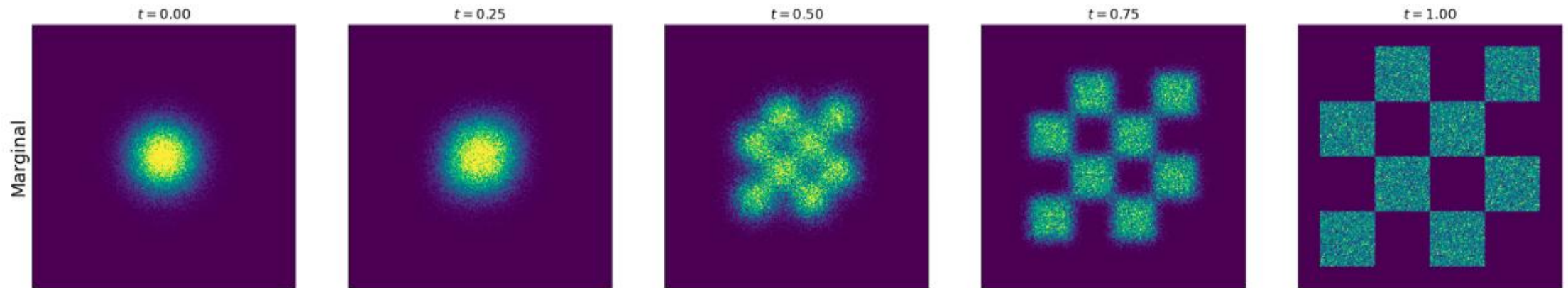
$$p_t(\cdot | z) \rightarrow p_t(\cdot)$$

# Comparison between conditional and marginal distribution

$X_t \sim p_t(\cdot | z)$ : Given data point  $z$ , what is the distribution at time  $t \in [0,1]$ ?



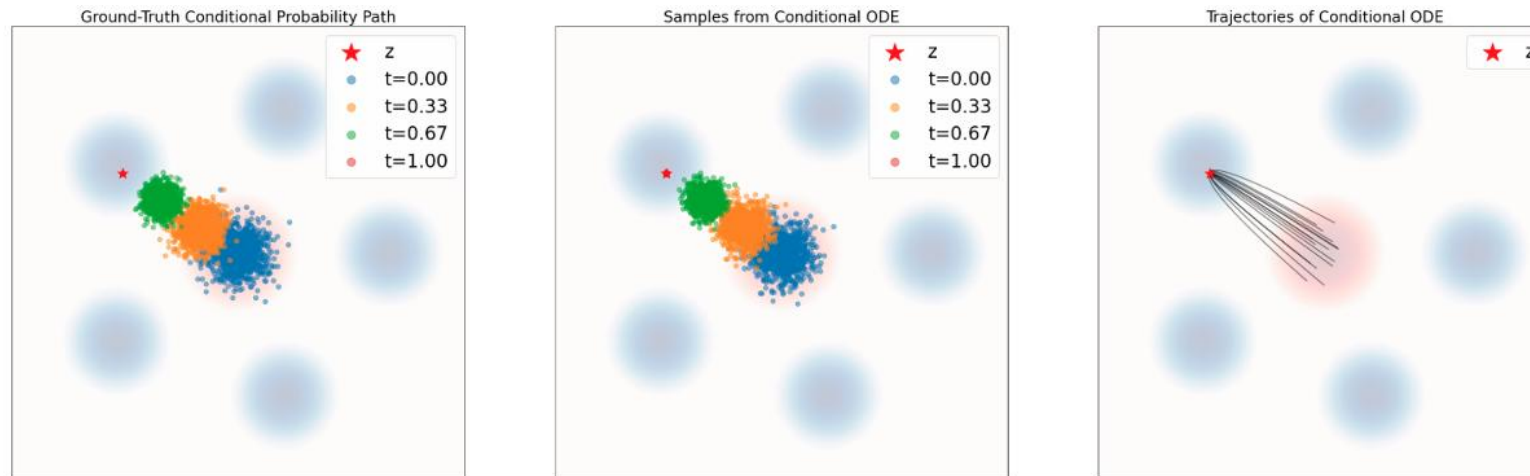
$X_t \sim p_t(\cdot)$ : what is the distribution at time  $t \in [0,1]$ ?



# Candidate for the conditional probability ( $p_t(\cdot | z)$ ) path

## We can imagine gaussian conditional path as a candidate

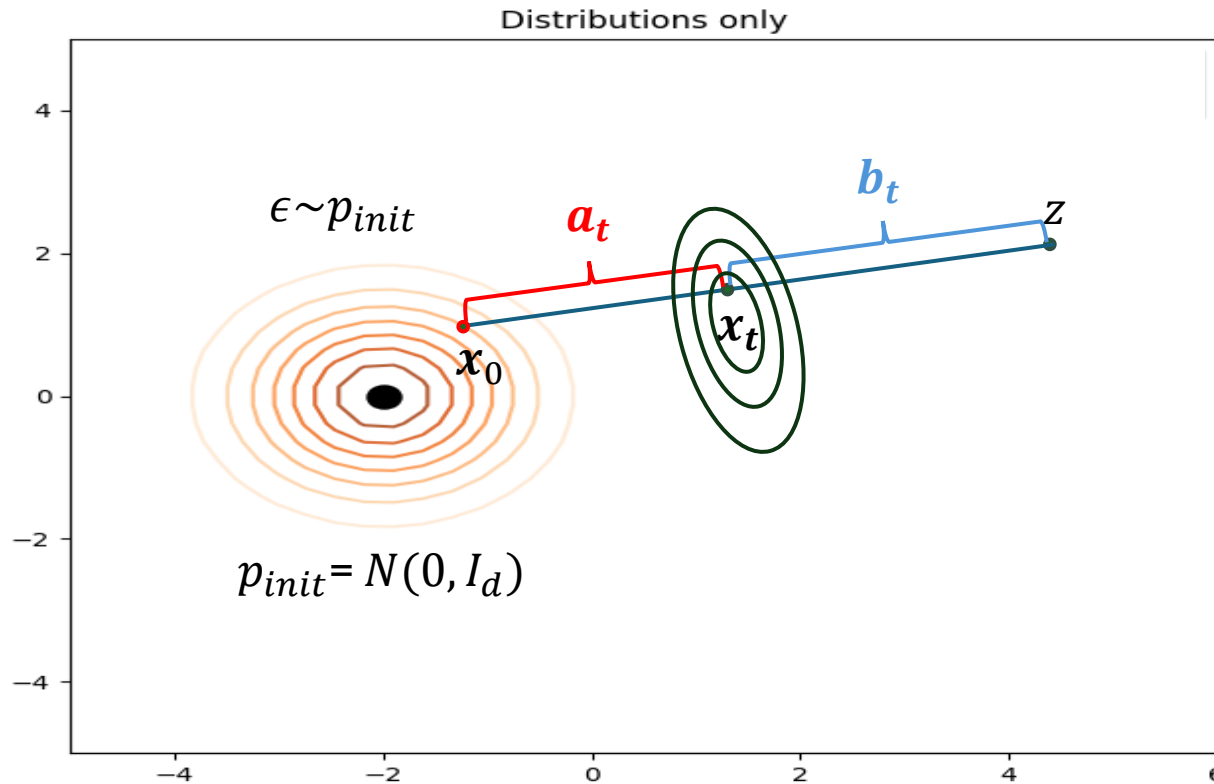
- Requirement:  $p_0(\cdot | z) = p_{init}$  and  $p_1(\cdot | z) = p_{final} = \delta_z$
- Assume that  $p_t(\cdot | z) = N(\alpha_t z, \beta_t^2 I_d)$ , where  $\alpha_0 = \beta_1 = 0$  and  $\alpha_1 = \beta_0 = 1$   
 $p_0(\cdot | z) = N(\alpha_0 z, \beta_0^2 I_d) = N(0, I_d)$  which is normal distribution  
 $p_1(\cdot | z) = N(\alpha_1 z, \beta_1^2 I_d) = N(z, 0)$  which always output  $z$
- Now we can obtain from  $\boxed{x_t = \alpha_t z + \beta_t \epsilon}$  from  $p_t(\cdot | z) = N(\alpha_t z, \beta_t^2 I_d)$  and  $\epsilon \sim N(0, I_d)$  by using reparameterization trick.



# Interpreting $x_t = \alpha_t z + \beta_t \epsilon$

$$p_0(\cdot | z) = N(0, I_d)$$

Since  $\epsilon$  is equal to  $x_0$ , we can interpret  $x_t$  as  $\alpha_t z + \beta_t x_0$



$$\begin{aligned} \text{> } x_t &= \alpha_t z + \beta_t \epsilon \\ &= \alpha_t z + \beta_t x_0 \end{aligned}$$

$$\begin{aligned} \text{> } (x_t - x_0) : (z - x_t) &= a_t : b_t \\ x_t &= (a_t z + b_t x_0) / (a_t + b_t) \end{aligned}$$

$$\alpha_t = \frac{a_t}{a_t + b_t}, \beta_t = \frac{b_t}{a_t + b_t}$$

$$\begin{aligned} \text{> } \psi_t(x_0 | z) &: \text{Position of } x_t \text{ obtained from } x_0 \\ \psi_t(x_0 | z) &= \alpha_t z + \beta_t x_0 \end{aligned}$$

If we use gaussian conditional path as a candidate,  
the flow  $\psi_t(x_0 | z)$  maps the  $x_t$  linearly!

# Conditional Gaussian vector field $u_t^{target}(x|z)$ from the prev. candidate

➤ Starting from definition of  $X_t$  and  $u_t^{target}$

$$\psi_t(x_0|z) = X_t = \alpha_t z + \beta_t x_0, \quad \frac{d}{dt} \psi_t(x_0|z) = u_t^{target}(\psi_t(x_0|z)|z)$$

➤ We can write  $\frac{d}{dt} \psi_t(x|z)$  and  $x$  in a following form:

$$\begin{aligned} \frac{d}{dt} \psi_t(x_0|z) &= \frac{d}{dt} (\alpha_t z + \beta_t x_0) \\ &= \dot{\alpha}_t z + \dot{\beta}_t x_0 \end{aligned}$$

➤ By combining above two equations, we can obtain:

$$\begin{aligned} u_t^{target}(\psi_t(x_0|z)|z) &= u_t^{target}(X_t|z) = \dot{\alpha}_t z + \dot{\beta}_t x_0 & x_0 &= (X_t - \alpha_t z)/\beta_t \\ &= \dot{\alpha}_t z + \dot{\beta}_t (X_t - \alpha_t z)/\beta_t \\ &= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} X_t \end{aligned}$$

# Conditional Gaussian vector field $u_t^{target}(x|z)$ from the prev. candidate

➤ Starting from definition of  $X_t$  and  $u_t^{target}$

$$\psi_t(x_0|z) = X_t = \alpha_t z + \beta_t x_0, \quad \frac{d}{dt} \psi_t(x_0|z) = u_t^{target}(\psi_t(x_0|z)|z)$$

➤ We can write  $\frac{d}{dt} \psi_t(x|z)$  and  $x$  in a following form:

Given  $z$ , the vector field (velocity) at position  $x$  can be written as:

$$u_t^{target}(x|z) = (\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t)z + \frac{\dot{\beta}_t}{\beta_t} x$$

➤ By combining above

$$\begin{aligned} u_t^{target}(\psi_t(x_0|z)|z) &= u_t^{target}(X_t|z) = \dot{\alpha}_t z + \dot{\beta}_t x_0 & x_0 &= (X_t - \alpha_t z)/\beta_t \\ &= \dot{\alpha}_t z + \dot{\beta}_t (X_t - \alpha_t z)/\beta_t \\ &= (\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t)z + \frac{\dot{\beta}_t}{\beta_t} X_t \end{aligned}$$

# What we had done so far: Obtaining $u_t^{target}(x|z)$

If we know  $z$ , we can find a conditional “vector field” that transport noise to  $z$

## 1. Define conditional probability path $p_t(\cdot | z)$ :

- $p_0(\cdot | z) = N(0, I_d)$  which is normal distribution
- $p_1(\cdot | z) = N(z, 0)$  which always output  $z$

## 2. Obtain $u_t^{target}(x|z)$ :

- Define  $\psi_t(x|z)$ : position of  $x$  defined from its initial position at time  $t$
- $\frac{d}{dt} \psi_t(x|z) = u_t^{target}(\psi_t(x|z)|z)$

$$\begin{aligned} p_t(\cdot | z) &= N(\alpha_t z, \beta_t^2 I_d) \\ &\downarrow \\ \psi_t(x_0 | z) &= X_t = \alpha_t z + \beta_t x_0 \\ &\downarrow \\ u_t^{target}(x|z) &= (\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t) z + \frac{\dot{\beta}_t}{\beta_t} x \\ u_t^{target}(x) &= f(x, t) \end{aligned}$$

## 3. However, we should know “marginal vector field”: $u_t^{target}(x)$

# Marginalization trick: obtaining $u_t^{target}(x)$ from $u_t^{target}(x|z)$

➤  $u_t^{target}(x|z)$ : A vector field of  $x$  when we know  $z$

- When we want to generate data (e.g., image), we want to find  $z$  from  $x$ .
- If we also need  $z$  when generating data, it is not useful..
- We need  $u_t^\theta(x)$  to generate data from noise.

$$p(z|x)p(x) = p(x|z)p(z) = p(x, z)$$

➤  $u_t^{target}(x) = E_{z \sim p(z|x)}[u_t^{target}(x|z)] = \int p(z|x) u_t^{target}(x|z) dz$

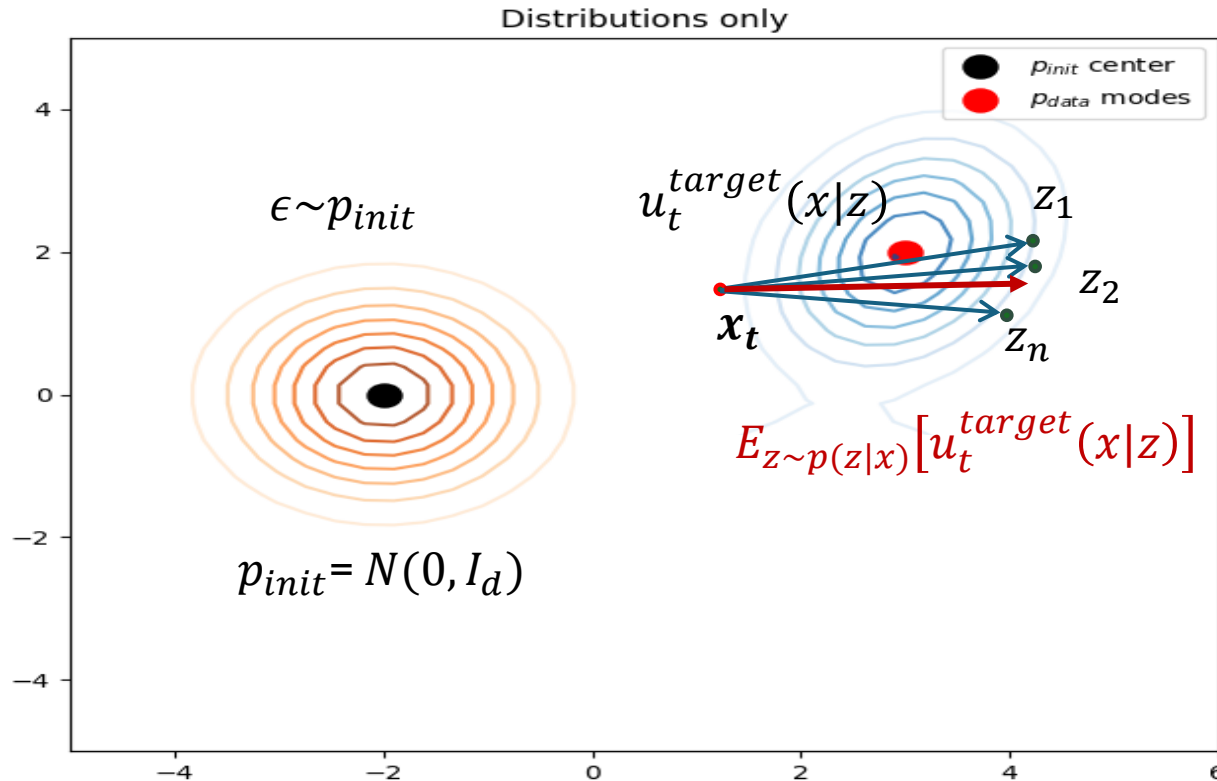
Is not tractable!

$$= \int u_t^{target}(x|z) \frac{p(x|z)p_{data}(z)}{p_t(x)} dz$$

➤  $X_0 \sim p_{init}, \frac{d}{dt} X_t = u_t^{target}(x), X_1 \sim p_{data}$

- We want  $u_t^\theta(x)$  that can represent  $u_t^{target}(x)$ !
- However,  $u_t^{target}(x)$  is hard to obtain. Can we use  $u_t^{target}(x|z)$  instead of  $u_t^{target}(x)$ ?

# Why do we use $E_{z \sim p(z|x)}$ instead of $E_{z \sim p(z)}$



- $u_t^{target}(x) = E_{z \sim p(z|x)}[u_t^{target}(x|z)]$
- Since we already know  $x_t$  (i.e., given  $x_t$ , what is possible  $z$  distribution), we should use  $p(z|x)$  instead of  $p(z)$ .
- If we use  $p(z)$ , we consider all possible  $z$  that are unlikely to have generated from the observed  $x_t$ .

# One remaining question..

## ➤ In flow matching...

$$u_t^{target}(x) = \int u_t^{target}(x|z) \frac{p(x|z)p_{data}(z)}{p_t(x)} dz$$

$$X_0 \sim p_{init}, \quad \frac{d}{dt} X_t = u_t^{target}(x), \quad X_1 \sim p_{data}$$

How can we be sure  $X_t \sim p_t$  — i.e.,  $X_t$  has distribution  $p_t$  ?

## ➤ Continuity Equation\*

- $X_t \sim p_t$  for all  $0 \leq t \leq 1$  if and only if

$$\frac{\partial}{\partial t} p_t(x) = \partial_t p_t(x) = -\nabla(p_t u_t^{target})(x) \text{ for all } x \in \mathcal{R}^d, 0 \leq t \leq 1$$

- Here, we will only get the intuition:
  - Left term,  $\partial_t p_t(x)$ : how much the probability  $p_t$  at  $x$  changes in time domain.
  - Right term,  $-\nabla(p_t u_t^{target})(x)$ : the net inflow of probability mass.

\* This is a specific case of Fokker-Planck equation. Check derivation of Fokker-Planck equation to understand the derivation process.

# Validation through continuity equation

We want to check whether  $p_t$  and  $u_t^{target}$  satisfy continuity equation.

$$X_t \sim p_t \text{ for all } 0 \leq t \leq 1 \text{ if and only if}$$
$$\frac{\partial}{\partial t} p_t(x) = \partial_t p_t(x) = -\nabla(p_t u_t^{target})(x) \text{ for all } x \in \mathcal{R}^d, 0 \leq t \leq 1$$

$$\partial_t p_t(x|z) = -\nabla(p_t(\cdot|z) u_t^{target}(\cdot|z))(x)$$

$$\begin{aligned} \partial_t p_t(x) &= \partial_t \int p_t(x|z) p_{data}(z) dz = \int \partial_t p_t(x|z) p_{data}(z) dz & u_t^{target}(x) &= \int u_t^{target}(x|z) \frac{p(x|z) p_{data}(z)}{p_t(x)} dz \\ &= \int -\nabla(p_t(\cdot|z) u_t^{target}(\cdot|z))(x) p_{data}(z) dz = -\nabla \int p_t(x|z) u_t^{target}(x|z) p_{data}(z) dz \\ &= -\nabla \int \frac{p_t(x)}{p_t(x)} p_t(x|z) u_t^{target}(x|z) p_{data}(z) dz \\ &= -\nabla p_t(x) \int u_t^{target}(x|z) \frac{p_t(x|z) p_{data}(z)}{p_t(x)} dz \\ &= -\nabla[p_t(x) u_t^{target}(x)] \end{aligned}$$

# Training objective: minimizing difference between $u_t^{target}(x)$ and $u_t^\theta(x)$

## ➤ Flow matching loss:

$$u_t^{target}(x) = \int u_t^{target}(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz$$

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t \sim \text{uni}, x \sim p_t} [\|u_t^\theta(x) - u_t^{target}(x)\|^2]$$

$$= \mathbb{E}_{t \sim \text{uni}, x \sim p_t} [\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{target}(x) + \|u_t^{target}(x)\|^2]$$

$$= \mathbb{E}_{t \sim \text{uni}, x \sim p_t} [\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t \sim \text{uni}, x \sim p_t} [u_t^\theta(x)^T u_t^{target}(x)] + \overset{C_1}{\mathbb{E}_{t \sim \text{uni}, x \sim p_t} [\|u_t^{target}(x)\|^2]}$$

$$= \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x)\|^2] - 2\boxed{\mathbb{E}_{t \sim \text{uni}, x \sim p_t} [u_t^\theta(x)^T u_t^{target}(x)]} + C_1$$

$$\overset{\text{red}}{\mathbb{E}_{t \sim \text{uni}, x \sim p_t} [u_t^\theta(x)^T u_t^{target}(x)]} = \int_0^1 \int p_t(x) u_t^\theta(x)^T u_t^{target}(x) dx dt$$

$$= \int_0^1 \int p_t(x) u_t^\theta(x)^T \int u_t^{target}(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz dx dt$$

$$= \int_0^1 \int \int u_t^\theta(x)^T u_t^{target}(x|z) p_t(x|z) p_{data}(z) dz dx dt$$

$$= \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [u_t^\theta(x)^T u_t^{target}(x|z)]$$

# Training objective: minimizing difference between $u_t^{target}(x)$ and $u_t^\theta(x)$

## ➤ Flow matching loss:

$$\begin{aligned}\mathcal{L}_{FM}(\theta) &= \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [u_t^\theta(x)^T u_t^{target}(x|z)] + C_1 \\&= \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{target}(x|z)] + C_1 \\&= \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{target}(x|z) + \underbrace{\|u_t^{target}(x|z)\|^2}_{\text{red}} - \|u_t^{target}(x|z)\|^2_{\text{red}}] + C_1 \\&= \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [\underbrace{\|u_t^\theta(x) - u_t^{target}(x|z)\|^2}_{\text{green}} - \|u_t^{target}(x|z)\|^2_{\text{red}}] + C_1 \\&= \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x) - u_t^{target}(x|z)\|^2] - \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [\|u_t^{target}(x|z)\|^2] + C_1 \\&= \mathbb{E}_{t \sim \text{uni}, z \sim p_{data}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x) - u_t^{target}(x|z)\|^2] + C_2 + C_1 \\&= \mathcal{L}_{CFM}(\theta) + C\end{aligned}$$

# Algorithm for Flow Matching Training

➤ Train neural network  $u_t^\theta$  using data sampled from  $z \sim p_{data}$

For each mini-batch of data

1. Sample a data  $z \sim p_{data}$
2. Sample a random time  $t \sim \text{uniform}[0,1]$
3. Sample noise  $\epsilon \sim N(0, I_d)$
4. Set  $x$  from  $x \sim p_t(\cdot | z)$  **we may choose  $X_t = \alpha_t z + \beta_t \epsilon$  if we assume conditional gaussian distribution.**
5. Compute  $\mathcal{L}_{CFM}(\theta) = \|u_t^\theta(x) - u_t^{target}(x|z)\|^2 = \left\| u_t^\theta(x) - \left( \dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \dot{\alpha}_t \right) z - \frac{\dot{\beta}_t}{\beta_t} x \right\|^2$
6. Update  $\theta \leftarrow \nabla \mathcal{L}_{CFM}(\theta)$

$$u_t^{target}(x|z) = (\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \dot{\alpha}_t) z + \frac{\dot{\beta}_t}{\beta_t} x$$

If we set  $\alpha_t = t$  and  $\beta_t = 1 - t$ , we can simplify

$$\mathcal{L}_{CFM}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2$$

$$\begin{aligned} &= \left\| u_t^\theta(x) - \left( \dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \dot{\alpha}_t \right) z - \frac{\dot{\beta}_t}{\beta_t} x \right\|^2 \\ &= \left\| u_t^\theta(x) - (\dot{\alpha}_t - \cancel{\frac{\dot{\beta}_t}{\beta_t} \dot{\alpha}_t}) z - \cancel{\frac{\dot{\beta}_t}{\beta_t}} (\alpha_t z + \beta_t \epsilon) \right\|^2 \\ &= \|u_t^\theta(\alpha_t z + \beta_t \epsilon) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2 \end{aligned}$$

# Algorithm: Sampling from a Flow Matching with Euler method

---

Now we have  $u_t^\theta$  that fits to  $u_t^{target}$

➤ With neural network vector field  $u_t^\theta$  and number of steps  $n$ :

1. Set  $t = 0$
2. Set step size  $h = \frac{1}{n}$
3. Sample  $X_0$  from  $p_{init}$
4. For  $i = 1, \dots, n$
5.      $X_{t+h} = X_t + hu_t^\theta(X_t)$
6.     Update  $t \leftarrow t + h$
7. End for
8. Return  $X_1$

# Numerical way to solve ODE

---

From the definition of derivative:

$$\frac{d}{dt}X_t = \frac{X_{t+h} - X_t}{h} = u_t(X_t)$$

We can obtain next step  $X_{t+h}$ :

$$X_{t+h} = X_t + hu_t(X_t)$$

Another way to estimate next step trajectory (Heun's method):

- First guess of next step trajectory:

$$X'_{t+h} = X_t + hu_t(X_t)$$

- Use current vector field and first guess vector field to find better next step trajectory:

$$X_{t+h} = X_t + 0.5h(u_t(X_t) + u_t(X'_{t+h}))$$